Configurable Signed-Distance-Field-Aware Planning and Whole-Body Model Predictive Control for Heterogeneous Mobile Manipulators

Xiaochen Miao
Technical University of Munich
Munich, Germany
xiaochen.miao@tum.de

***Abstract*—Coordinated mobile manipulation requires planning and control software to accommodate different arm dimensions, base kinematics, collision geometry, and hardware interfaces without replacing the core algorithm. This report presents a configurable pipeline that generates base and Cartesian end-effector references with the Open Motion Planning Library (OMPL), queries a shared Euclidean signed distance field (ESDF), and tracks the references with whole-body model predictive control (WB-MPC). Robot-specific YAML and URDF descriptions select state dimensions, limits, kinematics, collision spheres, and either nonholonomic or holonomic base dynamics. The same planning–control path is evaluated on two structurally different mobile manipulators: an 8-DoF reduced Hello Robot Stretch and a 9-DoF mobile UR10. In 40.02-s and 60.00-s PyBullet trials, end-effector reference RMSE was 16.6 and 18.5 mm, respectively; all ESDF queries were valid and executed-state safety margins remained positive. Mean complete control-call time was 81.4 and 71.4 ms. The Stretch completed three of four tasks, while the mobile UR10 completed all four. These results demonstrate reuse across two arm/base combinations, while platform-specific mimic-joint and guarded hardware adapters remain explicit boundary components rather than assumptions embedded in the generic optimizer.**

***Index Terms*—mobile manipulation, model predictive control, configurable robotics, motion planning, Euclidean signed distance field, ROS 2**

I. Introduction

Mobile manipulators extend the local reach of a fixed arm by coordinating manipulation with base motion. A useful control framework must do more than solve one robot's equations: platforms vary in arm dimension, reach, joint semantics, footprint, and whether the base can translate laterally. Stretch, for example, combines a differential-drive base, lift, telescopic arm, and wrist [1]; the second platform in this repository combines a six-axis UR10 with a holonomic planar base. Both require whole-body collision reasoning, but they must not be forced into the same base dynamics or command mapping.

Reference planning and feedback control solve different parts of this problem. A sampling-based planner can discover a collision-free route around an obstacle, but its path does not by itself reject disturbances or enforce smooth input and state bounds. MPC can repeatedly optimize a dynamically feasible trajectory, but a short prediction horizon is a poor replacement for global geometric planning. ESDFs provide a common collision representation: OMPL uses distance queries for state validity, while MPC uses distances and gradients in a differentiable penalty. The remaining challenge is to preserve these properties at the boundary to a physical robot, where stale feedback, solver overruns, command ownership, and driver-specific joint semantics matter.

This report asks: *How can OMPL reference planning, ESDF collision information, and WB-MPC be reused across mobile manipulators with different dimensions and base kinematics while retaining explicit platform interfaces?* The concrete contributions are:

- a YAML/URDF-parameterized WB-MPC model whose dimension, limits, kinematics, collision geometry, and base dynamics are selected per robot;
- OMPL base and Cartesian end-effector references with conservative ESDF validity checks and a squared-hinge MPC collision cost;
- validation of the shared pipeline on nonholonomic Stretch and holonomic mobile-UR10 configurations; and
- a default-shadow deployment adapter with preflight validation, velocity and acceleration limiting, a 50-Hz watchdog, recoverable holds, and latched stops for the currently integrated real Stretch.

Unlike a hardware-performance paper, this report does not claim autonomous closed-loop WB-MPC execution on the real robot. It evaluates the complete planner and controller in PyBullet and separately reports the real state and low-amplitude command-interface validation already recorded in the repository. This separation keeps cross-platform controller feasibility distinct from physical safety and does not imply that the Stretch adapter directly supports UR10 hardware.

II. Related Work

A. Mobile-Manipulator Planning and Control Paradigms

Conventional mobile-manipulation pipelines often decompose a task into base navigation followed by arm manipulation. This separation reduces the dimension of each planning problem and permits reuse of established navigation and manipulator controllers, but it discards base–arm redundancy and can produce conservative or suboptimal whole-body motion when the two subsystems must cooperate in constrained environments.

Kinematic whole-body control instead treats the base and arm as one redundant chain and resolves task-space commands

at the velocity level. Seraji extended configuration control to holonomic and nonholonomic mobile manipulators, incorporating base constraints and auxiliary tasks in a unified kinematic formulation [2]. De Luca *et al.* further considered nonholonomic mobile manipulators with steering wheels and used task-Jacobian null-space optimization and feedback linearization to determine otherwise unspecified steering motions [3]. These approaches are computationally efficient, but primarily resolve instantaneous kinematic tasks rather than predicting the coupled system evolution over a finite horizon.

Sampling-based predictive control provides another route to coupled and reactive motion. MPPI updates a nominal control sequence from the costs of sampled trajectory rollouts and can therefore accommodate complex or nonsmooth objectives without differentiating the complete cost. STORM demonstrated GPU-parallel sampling MPC for a 7-DoF manipulator [4], while Pezzato *et al.* extended GPU-parallel rollouts to navigation, contact-rich manipulation, and high-dimensional whole-body control [5]. RAMP combined an MPPI variant with an implicit configuration-space SDF for reactive obstacle avoidance [6]. Such sampling-based methods are flexible, but whole-body collision avoidance requires rollout costs across samples, horizon nodes, and represented robot bodies, commonly motivating substantial GPU parallelism. The present framework instead exploits ESDF gradients within a gradient-based model predictive controller.

B. Whole-Body Model Predictive Control

Whole-body MPC extends coordinated control beyond instantaneous kinematic resolution by optimizing the coupled base–arm evolution over a finite horizon. It repeatedly solves a finite-horizon problem from the latest state and executes only the first portion of the optimized sequence, allowing tracking, effort, smoothness, bounds, and collision terms to be treated together.

Whole-body MPC also couples manipulation with platform stability. Minniti *et al.* formulated manipulation, balancing, and interaction as one finite-horizon optimal-control problem for an inherently unstable ball-balancing mobile manipulator. Their formulation predicts how end-effector tracking and contact-force decisions affect the future balance of the system, and was validated in hardware pose-tracking and door-opening experiments [7]. This work establishes the value of jointly optimizing platform and manipulator behavior, but focuses on dynamic stabilization of a ballbot. The present work applies a unified predictive structure to kinematically different wheeled mobile manipulators. Enabling online whole-body collision avoidance within this structure additionally requires an environmental representation that can be queried efficiently at many prediction states, motivating the distance-field methods discussed next.

C. Distance-Field-Based Collision Avoidance

Conventional collision checking evaluates the geometric relationship between robot links and environmental obstacles explicitly. Depending on the environment representation, this may involve repeated collision or distance queries between robot meshes or collision primitives and individual obstacle geometries [8]. Such pairwise geometric queries can become computationally expensive when they are repeatedly evaluated for multiple robot links and prediction states within an online optimization loop.

Distance-field representations instead encode obstacle proximity independently of the robot geometry. ESDFs provide continuous clearance values and local spatial gradients, making them suitable for optimization-based motion generation. Voxblox demonstrated incremental ESDF construction from a TSDF for onboard planning [9], while nvblox accelerates dense incremental mapping on a GPU [10].

Distance fields have consequently been integrated into reactive motion-generation systems. Gaertner *et al.* used a dynamically generated ESDF for static collision checking and moving-obstacle models for dynamic scenes within a legged-robot MPC [11]. Chiu *et al.* incorporated an environment SDF into whole-body MPC and demonstrated collision-aware locomotion and manipulation while caching distance and gradient information to reduce environment-query overhead [12]. Both studies were validated on quadrupedal robot platforms: the former focused on legged locomotion, whereas the latter extended collision-aware MPC to legged mobile manipulation. They demonstrate the benefits of distance fields for reactive collision avoidance, but do not directly address a shared whole-body planning and control framework configurable for wheeled mobile manipulators with different arm dimensions and base kinematics.

III. Method

A. Robot-Parameterized Control Framework

The framework accepts a URDF and a composed YAML robot profile. The URDF defines the joints and links, kinematic tree, frame transforms, joint types, and arm joint-position limits. The profile selects the controller joint ordering and task frames from that model, and supplies controller-level choices not encoded by the URDF: the base dynamics, base workspace and velocity/input bounds, collision-sphere approximation, horizon, and cost weights. Pinocchio and CasADi combine these inputs to construct a robot-dimensioned symbolic OCP, from which acados generates a robot-specific solver. Changing the arm dimension, base dynamics, or number of collision spheres regenerates the solver but does not change the construction procedure.

At runtime, a control goal is converted into a reference horizon, the ESDF supplies local obstacle information, and the generated solver jointly optimizes base and arm motion. A platform adapter then converts the optimized trajectory into robot commands. Figure 1 summarizes both offline robot specialization and the online feedback-control path.

The following subsections define the dynamics, ESDF collision model, OCP, and task-conditioned reference generation used in this pipeline.

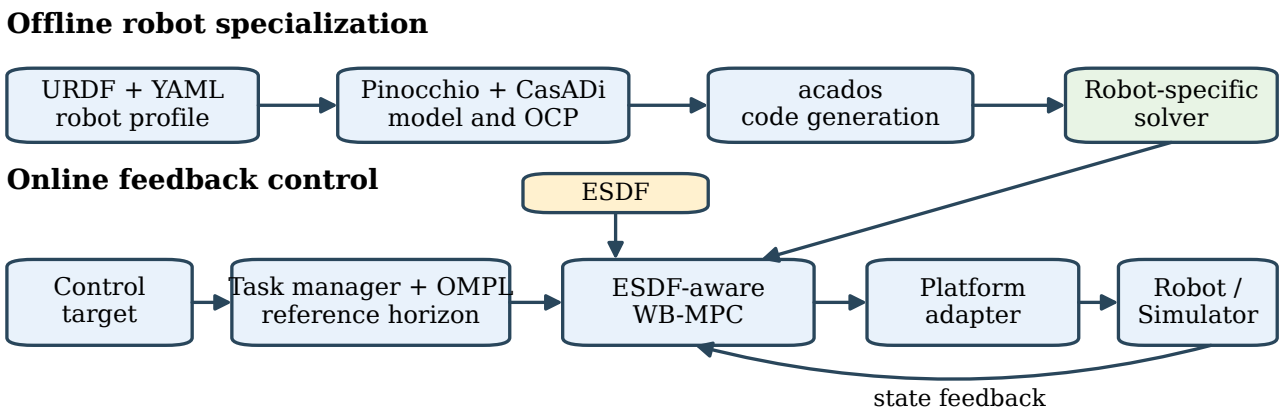

Fig. 1. Robot-parameterized solver generation and online control flow. A URDF and YAML profile specialize the generated solver; control targets and ESDF data then enter the shared WB-MPC path before platform-specific command mapping.

B. Model Predictive Control

1) Robot-Parameterized Dynamics: Each robot profile selects the base dynamics branch: an omnidirectional base uses a planar double-integrator model, whereas a nonholonomic base can use the embedded no-slip dynamics derived below. The URDF specializes the arm dimension and kinematics, and all arm joints use acceleration-controlled double-integrator dynamics.

For a robot with n_a independent arm coordinates, the optimizer constructs

$$\boldsymbol{q} = \begin{bmatrix} x & y & \theta & \boldsymbol{q}_a^\top \end{bmatrix}^\top, \quad \boldsymbol{x} = \begin{bmatrix} \boldsymbol{q}^\top & \boldsymbol{v}^\top \end{bmatrix}^\top, \tag{1}$$

The corresponding continuous-time state equation is written generically as

$$\dot{\boldsymbol{x}} = f(\boldsymbol{x}, \boldsymbol{u}) = \begin{bmatrix} \dot{\boldsymbol{q}}^\top & \dot{\boldsymbol{v}}^\top \end{bmatrix}^\top, \tag{2}$$

where (x, y, θ) is the map-frame planar base pose. Let n_q, n_u, and n_x denote the dimensions of the generalized-coordinate vector, optimizer input, and state vector, respectively. The planar base contributes three coordinates, while the arm contributes n_a independent coordinates; hence, $n_q = 3 + n_a$. Because the optimizer assigns one acceleration input to each generalized coordinate, $n_u = n_q$. The state stacks generalized positions and velocities, giving $n_x = 2n_q$. These dimensions describe the internal optimization model rather than the number of physical actuators. Joint names, bounds, link frames, tool and base links, collision primitives, and URDF are read from the selected robot profile.

For an omnidirectional base, every coordinate follows the common double-integrator model $\dot{\boldsymbol{q}} = \boldsymbol{v}$, $\dot{\boldsymbol{v}} = \boldsymbol{u}$, and therefore

$$\dot{\boldsymbol{x}}_{\text{omni}} = \begin{bmatrix} \boldsymbol{v}^\top & \boldsymbol{u}^\top \end{bmatrix}^\top. \tag{3}$$

For a nonholonomic base in dynamics mode, the same public world-frame state $[v_x, v_y, \omega]$ is retained, but the base part is replaced by a no-slip model. Its forward and lateral components are

$$v_f = \cos\theta\, v_x + \sin\theta\, v_y, \tag{4}$$

$$v_{\text{lat}} = -\sin\theta\, v_x + \cos\theta\, v_y = 0. \tag{5}$$

The first two world-frame acceleration variables are similarly projected to $a_f = \cos\theta\, a_x + \sin\theta\, a_y$. The embedded base dynamics are

$$\dot{x} = v_f \cos\theta, \qquad \dot{y} = v_f \sin\theta, \qquad \dot{\theta} = \omega, \tag{6}$$

$$\dot{v}_x = a_f \cos\theta - \omega v_f \sin\theta, \tag{7}$$

$$\dot{v}_y = a_f \sin\theta + \omega v_f \cos\theta, \tag{8}$$

$$\dot{\omega} = \alpha. \tag{9}$$

These equations define the two base-dynamics branches selected by the robot profile.

2) ESDF Collision Avoidance: The environment representation is independent of the robot model. Each robot profile instead attaches a set of collision spheres to selected links. Given configuration $\boldsymbol{q}_k$, forward kinematics determines the center $\boldsymbol{c}_j(\boldsymbol{q}_k)$ of sphere j. Define the configuration-dependent clearance as $d_j(\boldsymbol{q}) \triangleq d(\boldsymbol{c}_j(\boldsymbol{q}))$, where the ESDF returns both the signed distance and its spatial gradient.

The exported ESDF is sampled on a regular three-dimensional grid. A query point is first located within a grid cell. The implementation then uses its relative position along the three coordinate axes to trilinearly interpolate both the distance and the stored gradient from the cell's eight corners. The resulting signed distance and spatial gradient are denoted by $\widehat{d}(\boldsymbol{c})$ and $\widehat{\boldsymbol{n}}(\boldsymbol{c})$, respectively. Under the conservative validity policy used in the experiments, a query is accepted only if it lies inside the grid and all eight interpolation corners are marked valid. The interpolation supplies a continuous local distance–gradient pair to the MPC update.

Because the gridded ESDF is queried outside the generated acados solver, its local values are updated from the warm-start

trajectory before every MPC solve. At shooting node k, the first-order Taylor model around the sphere center $\boldsymbol{c}_j(\bar{\boldsymbol{q}}_k)$ is

$$\begin{aligned}\widetilde{d}_{j,k}(\boldsymbol{q}) = \widehat{d}(\boldsymbol{c}_j(\bar{\boldsymbol{q}}_k)) \\ + \widehat{\boldsymbol{n}}(\boldsymbol{c}_j(\bar{\boldsymbol{q}}_k))^\top \left(\boldsymbol{c}_j(\boldsymbol{q}) - \boldsymbol{c}_j(\bar{\boldsymbol{q}}_k)\right).\end{aligned} \tag{10}$$

Thus, the solver retains the robot-specific forward-kinematics expression $\boldsymbol{c}_j(\boldsymbol{q})$ but replaces the external map query with a local affine distance-field model. The way this model produces a gradient with respect to the robot coordinates can be seen directly from its differential. Define the point Jacobian $J_{\boldsymbol{c}_j}(\boldsymbol{q}) \triangleq \partial \boldsymbol{c}_j(\boldsymbol{q})/\partial \boldsymbol{q}$. A small configuration change $\delta \boldsymbol{q}$ moves the sphere center by

$$\delta \boldsymbol{c}_j = J_{\boldsymbol{c}_j}(\boldsymbol{q})\, \delta \boldsymbol{q}. \tag{11}$$

The ESDF distance and gradient evaluated at the warm-start center remain constant during one solve. Differentiating Eq. (10) therefore gives

$$\begin{aligned}\delta \widetilde{d}_{j,k} &= \widehat{\boldsymbol{n}}(\boldsymbol{c}_j(\bar{\boldsymbol{q}}_k))^\top \delta \boldsymbol{c}_j \\ &= \widehat{\boldsymbol{n}}(\boldsymbol{c}_j(\bar{\boldsymbol{q}}_k))^\top J_{\boldsymbol{c}_j}(\boldsymbol{q})\, \delta \boldsymbol{q}.\end{aligned} \tag{12}$$

By the gradient definition $\delta \widetilde{d}_{j,k} = (\nabla_{\boldsymbol{q}} \widetilde{d}_{j,k})^\top \delta \boldsymbol{q}$, comparison of the two expressions yields

$$\nabla_{\boldsymbol{q}} \widetilde{d}_{j,k}(\boldsymbol{q}) = J_{\boldsymbol{c}_j}(\boldsymbol{q})^\top \widehat{\boldsymbol{n}}(\boldsymbol{c}_j(\bar{\boldsymbol{q}}_k)). \tag{13}$$

In physical terms, each column of the point Jacobian describes how one robot coordinate moves the sphere center, while its dot product with $\widehat{\boldsymbol{n}}(\boldsymbol{c}_j(\bar{\boldsymbol{q}}_k))$ measures how strongly that motion increases or decreases the local obstacle distance. The code does not explicitly construct this matrix product or query a new ESDF gradient inside acados. CasADi obtains it by automatically differentiating the local distance model; the interpolated gradient remains fixed until the next MPC update.

For sphere radius r_j and configured safety distance d_s, the geometric margin at prediction node k is

$$m_{j,k} = d(\boldsymbol{c}_j(\boldsymbol{q}_k)) - r_j - d_s. \tag{14}$$

Inside the solver, d is replaced by the Taylor model in Eq. (10), giving $\widetilde{m}_{j,k} = \widetilde{d}_{j,k}(\boldsymbol{q}_k) - r_j - d_s$. The collision condition is implemented as a soft cost rather than a hard inequality constraint. Specifically, the implementation uses a smooth relaxation of the hinge violation $\max(0, -m)$ and then squares it:

$$v_\varepsilon(m) = \frac{1}{2}\left(\sqrt{m^2 + \varepsilon^2} - m\right), \tag{15}$$

$$\phi(m) = \frac{1}{2}\mu\, v_\varepsilon(m)^2 = \frac{\mu}{8}\left(\sqrt{m^2 + \varepsilon^2} - m\right)^2. \tag{16}$$

As $\varepsilon \to 0$, $\phi(m)$ converges to $\frac{1}{2}\mu \max(0, -m)^2$. The total collision cost over the prediction horizon is then

$$J_{\text{obs}} = \sum_{k=0}^{N-1} \sum_{j=1}^{n_c} \phi(\widetilde{m}_{j,k}), \tag{17}$$

where n_c denotes the number of collision spheres attached to the robot. The Stretch and mobile-UR10 profiles supply 34 and 93 link-attached spheres, respectively. At every MPC update, ESDF values and gradients are queried along the warm-start trajectory and supplied to all shooting nodes. The formulation is unchanged when the number or placement of collision primitives changes.

3) Whole-Body MPC Formulation: At each cycle the controller solves

$$\min_{\boldsymbol{x}_{0:N}, \boldsymbol{u}_{0:N-1}} \quad \sum_{k=0}^{N-1} \ell(\boldsymbol{x}_k, \boldsymbol{u}_k, \boldsymbol{r}_k) + \ell_N(\boldsymbol{x}_N, \boldsymbol{r}_N) + J_{\text{obs}} \tag{18}$$

$$\text{s.t.} \quad \boldsymbol{x}_{k+1} = f_d(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad \underline{\boldsymbol{x}} \le \boldsymbol{x}_k \le \overline{\boldsymbol{x}}, \quad \underline{\boldsymbol{u}} \le \boldsymbol{u}_k \le \overline{\boldsymbol{u}}. \tag{19}$$

To make the tracking metric explicit, let $\boldsymbol{e}_k = h(\boldsymbol{x}_k) - \boldsymbol{r}_k$ collect the base-pose, end-effector-pose, base-velocity, and end-effector-velocity residuals. Heading error is wrapped on $SO(2)$, and end-effector orientation error is computed with the $SO(3)$ logarithm. The noncollision stage and terminal losses are

$$\begin{aligned}\ell(\boldsymbol{x}_k, \boldsymbol{u}_k, \boldsymbol{r}_k) &= \tfrac{1}{2}\|\boldsymbol{e}_k\|^2_{\boldsymbol{W}_k} + \tfrac{1}{2}\|\boldsymbol{x}_k\|^2_{\boldsymbol{Q}_x} \\ &\quad + \tfrac{1}{2}\|\boldsymbol{u}_k\|^2_{\boldsymbol{R}_u} + \ell_{\text{arm}}(\boldsymbol{q}_k), \\ \ell_N(\boldsymbol{x}_N, \boldsymbol{r}_N) &= \tfrac{1}{2}\|\boldsymbol{e}_N\|^2_{\boldsymbol{W}_N}.\end{aligned} \tag{20}$$

Here $\|\boldsymbol{z}\|^2_{\boldsymbol{W}} \triangleq \boldsymbol{z}^\top \boldsymbol{W} \boldsymbol{z}$ defines the weighted squared Euclidean loss. The matrices $\boldsymbol{Q}_x$ and $\boldsymbol{R}_u$ penalize state and control effort, while ℓ_{arm} is the optional arm-extension shaping cost. Zero blocks in $\boldsymbol{W}_k$ deactivate residuals that are irrelevant to the current task. The ESDF term is included separately through J_{obs} in Eq. (17).

State and input box bounds are hard, whereas collision avoidance enters through the soft cost J_{obs}. Consequently, the OCP alone does not provide a hard collision-safety guarantee.

C. Task-Conditioned Reference Generation

The task manager maintains an ordered sequence of base and end-effector tasks and exposes one active task type $\tau \in \{\text{base}, \text{EE}\}$ to the controller. For a base task, the geometric path is planned in $SE(2)$ with the RRT* implementation provided by OMPL [13], [14]. The planner uses the ESDF state-validity callback to reject configurations that violate the configured footprint clearance, and RRT* rewires the sampling tree to reduce path cost. For an end-effector task, the current profile uses OMPL's RRTConnect planner in the bounded Cartesian workspace. After path simplification, let $\boldsymbol{\gamma}_\tau(s)$ denote the resulting geometric path parameterized by progress s. The reference horizon is generated from the measured path progress s_0 as

$$\begin{aligned}s_k &= \text{clip}(s_0 + v_\tau k \Delta t, 0, s_{\text{end}}), \\ \boldsymbol{r}_k &= \boldsymbol{\gamma}_\tau(s_k), \qquad k = 0, \ldots, N,\end{aligned} \tag{21}$$

where v_τ is the nominal progress rate. Thus $\boldsymbol{r}_0$ is the current desired reference, and the remaining samples populate the MPC prediction horizon. A waypoint task is the constant-path special case.

The task type selects the active residual blocks in Eq. (20). A base task activates $SE(2)$ pose and base velocity tracking and

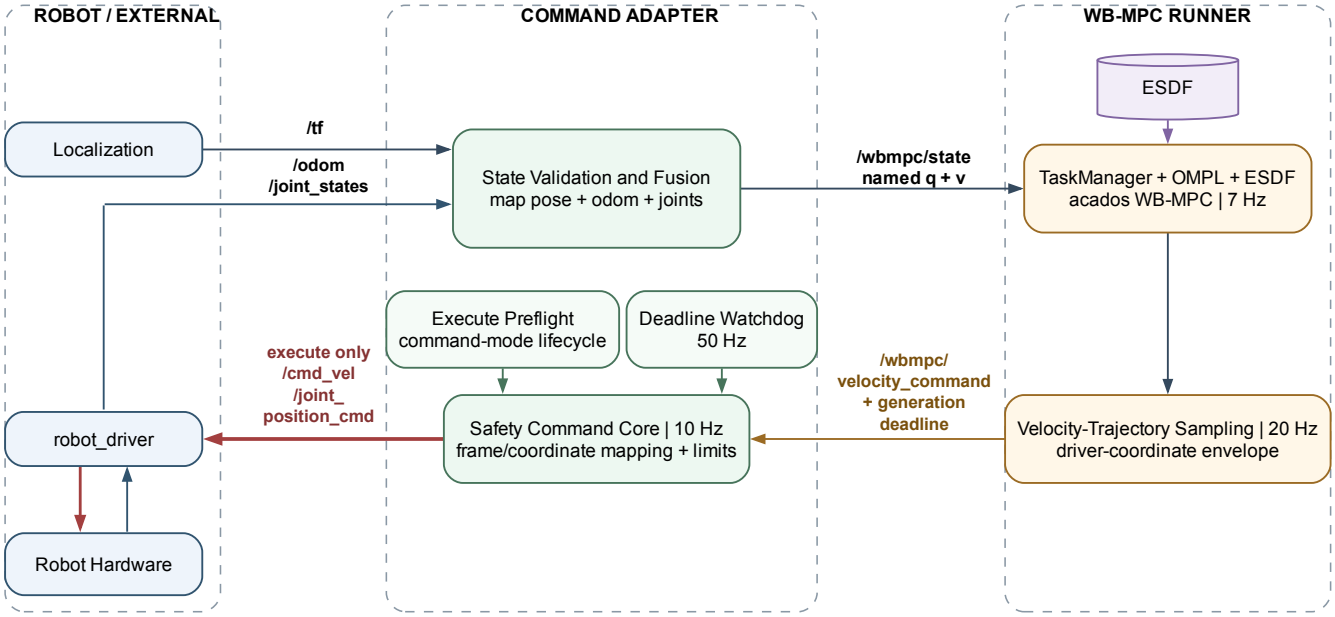

Fig. 2. Platform-abstract ROS 2 nodes and data flow for real-robot execution. The upper path carries feedback into WB-MPC, and the lower path returns the deadline-stamped command envelope. Red hardware outputs exist only with `execute=true`; diagnostic topics and driver-mode service calls are omitted for legibility.

can tighten the preferred arm extension. An end-effector task activates Cartesian position and, when requested, orientation and velocity tracking while allowing the optimizer to coordinate the mobile base and manipulator through whole-body optimal control. Residual masks set irrelevant coordinates to zero weight. Once the active task remains within its configured position and orientation tolerances for the hold period, the manager advances to the next task.

IV. System Implementation

This section describes the execution framework and adaptation strategy developed for real-robot deployment.

A. ROS 2 Nodes and Topic Topology

Figure 2 abstracts the real-robot execution path into a driver, a command adapter, and a WB-MPC runner. The `robot_command_adapter` is the only application node connected to the robot driver. It subscribes to odometry, joint feedback, the global-to-base transform, and the driver mode and safety states. Valid temporally aligned base and joint samples are combined into a named `sensor_msgs/JointState` on `/wbmpc/state`. Positions and velocities use the controller ordering, and the header stamp and frame identify the source time and global frame used by the controller.

The `wbmpc_runner` subscribes to this normalized state interface. It owns the task manager, OMPL planners, and WB-MPC instance; the ESDF supplies queries to both planning and control. A worker thread starts planning and optimization at 7 Hz so that state callbacks and the command publisher do not block on a solve. A separate 20-Hz timer samples the most recent valid velocity trajectory, converts it to the driver's coordinate representation, and publishes `/wbmpc/velocity_command`. The serialized envelope contains the velocity vector, solver generation, validity flag, source-state stamp, plan origin, and an absolute monotonic validity deadline. The runner and adapter publish JSON status records on `/wbmpc/status` and `/robot_command_adapter/status` for logging and diagnosis.

The adapter consumes the envelope and, in execute mode, publishes base velocity on `/cmd_vel` and a position-oriented joint command on `/joint_position_cmd` at 10 Hz. Consequently, the optimizer never owns a driver topic directly, and all feedback validation, command conversion, deadline enforcement, and driver-mode service calls remain at one ROS 2 boundary.

B. Forward Prediction and Deadline Hold

In simulation, the solver and simulator run sequentially in a blocking loop, so the simulated robot state does not evolve while the optimization is in progress. In real-robot deployment, however, the solver runs concurrently with the continuously moving robot, making compensation for this motion necessary.

The initial state used by the MPC is measured before optimization and does not account for the robot's motion during computation. Consequently, by the time the optimized command reaches the robot, it is based on an outdated state. The runner can compensate by advancing the measured state over

$$\Delta t_{\text{pred}} = a_{\text{state}} + \hat{t}_{\text{solve}} + t_{\text{act}}, \tag{22}$$

where a_{state} is the source age of the synchronized feedback, $\hat{t}_{\text{solve}}$ is an online estimate of planning and optimization time, and t_{act} is the configured dispatch and actuation latency. After each successful cycle, the computation-time estimate is updated as

$$\hat{t}^{+}_{\text{solve}} = \text{clip}\big((1-\alpha)\hat{t}_{\text{solve}} + \alpha t_{\text{solve}}, t_{\min}, t_{\max}\big)\,. \tag{23}$$

where t_{solve} is the solve time measured in the current cycle, $\hat{t}_{\text{solve}}$ is its estimate before the update, and the superscript $+$ denotes the updated estimate. The adaptation factor $\alpha \in [0,1]$ weights the latest measurement, while $t_{\min}$ and $t_{\max}$ bound the updated estimate. The current configuration sets $\alpha = 0.10$, initializes the estimate at 60 ms, clips it to $[20, 120]$ ms, adds 50 ms actuation latency, and rejects a prediction interval above 300 ms.

The initial condition imposed on the MPC is therefore

$$\begin{aligned} \boldsymbol{x}_0^{\text{MPC}} &= \Phi_{\Delta t_{\text{pred}}}(\boldsymbol{x}_{\text{meas}}, \boldsymbol{u}_{\text{last}})\,, \\ \boldsymbol{x}_{\text{meas}} &= \begin{bmatrix} \boldsymbol{q}_{\text{meas}}^{\top} & \boldsymbol{v}_{\text{meas}}^{\top} \end{bmatrix}^{\top}. \end{aligned} \tag{24}$$

where $\boldsymbol{u}_{\text{last}}$ is the most recently published acceleration and Φ_τ denotes propagation over duration τ using the same discrete robot dynamics and state/input bounds as the OCP. Without forward prediction, $\boldsymbol{x}_0^{\text{MPC}} = \boldsymbol{x}_{\text{meas}}$. The controller plans and solves from this initial state and the corresponding future task time.

Command validity is bounded independently of solver termination. The deadline of each optimization cycle also limits the lifetime of the active command. If no replacement is available before this deadline, the runner invalidates the active plan and issues a zero-velocity envelope. A late solution can be admitted under the configured policy, but its time origin and validity interval are reset to its completion time; an expired command interval is never revived.

The hardware adapter independently checks the envelope deadline using a monotonic clock. This check ensures that a timeout produces zero base velocity and an arm position hold even if the optimization thread stalls. A fresh valid command automatically releases this recoverable hold. In contrast, violations of state integrity, command mapping, device mode, or command ownership trigger a latched stop that requires a restart and new preflight before execution can resume.

V. Experimental Evaluation

A. Setup and Metrics

Two headless PyBullet trials ran the repository's canonical `stretch_esdf_offline_ompl_wbmpc.yaml` and `ur10_esdf_offline_ompl_wbmpc.yaml` profiles. Both used the same experiment runner, task-manager classes, ESDF implementation, MPC class, and 30-ms simulator step; selecting a different composed YAML changed the robot model and dimensions. The mobile-UR10 solver artifact was absent, so `recompile` was enabled for that trial without changing control parameters. Fully expanded configurations are archived with both raw NPZ files. The working tree was based on Git commit `58b32939cd4030ec9d2aec86003926bc575a4b59` and the machine used an Intel Core i9-14900HX CPU, Ubuntu 24.04, Python 3.11.15, CasADi 3.7.2, and Pinocchio 3.7.0; acados was compiled without OpenMP.

TABLE I
Evaluated Specializations of the Shared Framework

Property	Stretch	Mobile UR10
Optimized coordinates	8	9
Arm coordinates n_a	5	6
Base model	nonholonomic	holonomic
State dimension n_x	16	18
Collision spheres	34	93
Extra ESDF distance	0.07 m	0.10 m

TABLE II
Simulation Results for Two Robot Configurations

Metric	Stretch	Mobile UR10
Duration / samples	40.02 s / 1334	60.00 s / 2000
Base position RMSE	20.5 mm	14.1 mm
Base yaw RMSE	6.55 mrad	3.31 mrad
EE position RMSE	16.6 mm	18.5 mm
Final task-target error	366 mm	20.4 mm
Minimum surface clearance	73.2 mm	101.5 mm
Minimum safety margin	3.22 mm	1.47 mm
Invalid ESDF queries	0	0
Controller mean / P95	81.4 / 141.0 ms	71.4 / 80.4 ms
Controller maximum	785.3 ms	799.3 ms
Calls above 120 ms	100 (7.50%)	21 (1.05%)
Solver status 0 / 2	1296 / 38	1998 / 2
Fallbacks / command clips	0 / 0	1 / 0
Completed tasks	3 / 4	4 / 4

Table I summarizes the two evaluated robot profiles. The Stretch trial exercises the nonholonomic dynamics branch, whereas the mobile-UR10 trial exercises the omnidirectional branch.

Position RMSE is computed against the active time-varying reference and yaw error is wrapped to $[-\pi, \pi]$. Timing encloses the complete control call. The 120-ms count is the real Stretch profile's validity window and is used as a common timing reference, not as a claimed UR10 hardware deadline. Since robot geometry and tuning differ, Table II demonstrates cross-platform execution rather than ranking the robots.

B. Cross-Platform Tracking

The Stretch base task occupied 314 samples through 9.42 s, with 20.5-mm position and 6.55-mrad yaw RMSE. Its remaining 1,020 samples contained EE references with 16.6-mm position RMSE. Three tasks completed; the fourth had started but its final target was still 0.366 m away at the fixed 40-s stop. During EE tasks the base moved 0.537 m from its earlier goal, confirming simultaneous whole-body behavior. Equation (5) remained the active base dynamics rather than an assumption made by the common planner.

The mobile UR10 base task used 300 samples through 9.00 s and achieved 14.1-mm position and 3.31-mrad yaw RMSE; its final active base-reference error was 4.87 mm. Across 1,700 EE-reference samples, position RMSE was 18.5 mm. All four tasks completed and final error to the fourth task

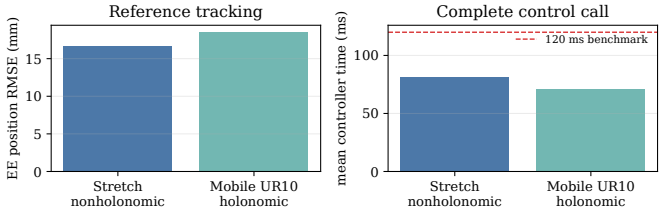

Fig. 3. Measured reference tracking and complete control-call time for the two configurations. Different geometry and tuning preclude a robot ranking.

target was 20.4 mm. The holonomic double-integrator branch, six direct arm coordinates, 93-sphere model, and different tool frame were instantiated from configuration while the execution path remained shared. Figure 3 summarizes the principal comparison.

Axis-wise EE RMSE further distinguishes the trials. Stretch produced 10.4, 9.91, and 8.34 mm in x, y, and z; the mobile UR10 produced 7.93, 3.70, and 16.3 mm. Thus the similar Euclidean RMSE values do not imply identical error structure: vertical tracking dominates the mobile-UR10 value. Error to the final *active reference sample* was 7.16 mm for Stretch and 16.4 mm for the mobile UR10. This differs from error to the final task target because Stretch's time-varying reference had not yet reached the fourth goal when its trial stopped. Figure 4 exposes both base paths and the complete EE error histories rather than only their aggregate RMSE.

C. ESDF Clearance

All ESDF queries were valid on both robots. Minimum node-0 surface clearance, defined as distance minus sphere radius, was 73.2 mm for Stretch and 101.5 mm for the mobile UR10. After subtracting their separately configured 70- and 100-mm safety distances, the minimum executed-state margins were positive but small: 3.22 and 1.47 mm. The pre-optimization warm-start horizon reached -24.5 and -83.9 mm, respectively. These values do not mean that the executed state collided, but they expose the limitation of a shifted warm start and soft collision cost. Post-solve horizon margins should be logged on every status, not only in the maximum-iteration acceptance gate.

D. Solver and Runtime Timing

Complete controller time averaged 81.4 ms for Stretch and 71.4 ms for the mobile UR10. Their P95 values were 141.0 and 80.4 ms, but isolated maxima of 785.3 and 799.3 ms show long tails on both. Stretch had 38 maximum-iteration status-2 cycles, all accepted by the finite, valid, nonnegative candidate test. The mobile UR10 had two: one was accepted and one produced the sole zero-velocity fallback. No command was clipped. The 93-sphere UR10 profile spent more mean time in parameter update and ESDF linearization (51.3 and 37.4 ms) than Stretch (30.6 and 15.9 ms), yet its easier OCP solves averaged 8.74 rather than 41.6 ms. This decomposition shows that robot geometry and optimization difficulty affect different runtime components; configurability does not imply platform-independent timing.

TABLE III
RUNTIME AND SOLVER BREAKDOWN

Metric	Stretch	Mobile UR10
Controller median / P99 (ms)	53.6 / 700.6	68.6 / 120.3
OCP solve mean / P95 (ms)	41.6 / 100.2	8.74 / 16.0
Parameter update mean (ms)	30.6	51.3
ESDF linearization mean (ms)	15.9	37.4
Diagnostic overhead mean (ms)	6.73	10.2
SQP iterations mean / P95	32.0 / 77.7	5.15 / 10.0
SQP iterations maximum	500	500
Accepted nonzero status	38	1
Zero-velocity fallback	0	1

Table III gives the distribution and solver-iteration details. Parameter update includes all shooting-node parameters, and ESDF linearization is nested within that work; the rows must therefore not be summed as independent totals. The UR10's larger sphere set increases the largely deterministic parameter cost, whereas Stretch's difficult SQP cycles dominate its upper tail. In particular, the median-to-P99 jump from 53.6 to 700.6 ms for Stretch is much larger than the mobile UR10's 68.6 to 120.3 ms.

Figure 5 shows why both central tendency and tails are necessary. It also aligns each timing distribution with the executed-state collision margin over time. No causal relationship should be inferred directly from this plot: solver difficulty depends on tracking, active costs, bounds, and local ESDF geometry simultaneously.

E. Real-Interface Validation Evidence

Two repository validation records predate the simulation trial and exercise the real Stretch interface without claiming full closed-loop WB-MPC. The final synchronized state test produced 594 valid complete 11-D states at 30.0 Hz, exactly zero base/joint timestamp skew, 5.01-ms P99 state age, and 19.64-ms maximum age. A navigation plus streaming-position test recorded 335 joint-state samples with a 41.218-ms maximum receive interval while commanding simultaneous ± 0.01 m/s base motion and a 0.02-rad wrist-yaw excursion. The base returned within 0.670 mm and wrist yaw within 7.03 mrad. These tests validate ordering, signs, update continuity, and low-amplitude concurrency.

They do not validate autonomous collision avoidance or solver-to-hardware timing. In particular, the driver was observed not to provide the required stale-command watchdog, motivating the independent adapter watchdog. Table IV keeps demonstrated interface facts separate from unperformed full-system claims.

VI. DISCUSSION

The two trials provide executable evidence that the central planning and control framework is not specific to Stretch. Without changing the common experiment runner or MPC class, it instantiated different state dimensions, arm chains, tool frames, collision-sphere sets, clearances, and base dynamics.

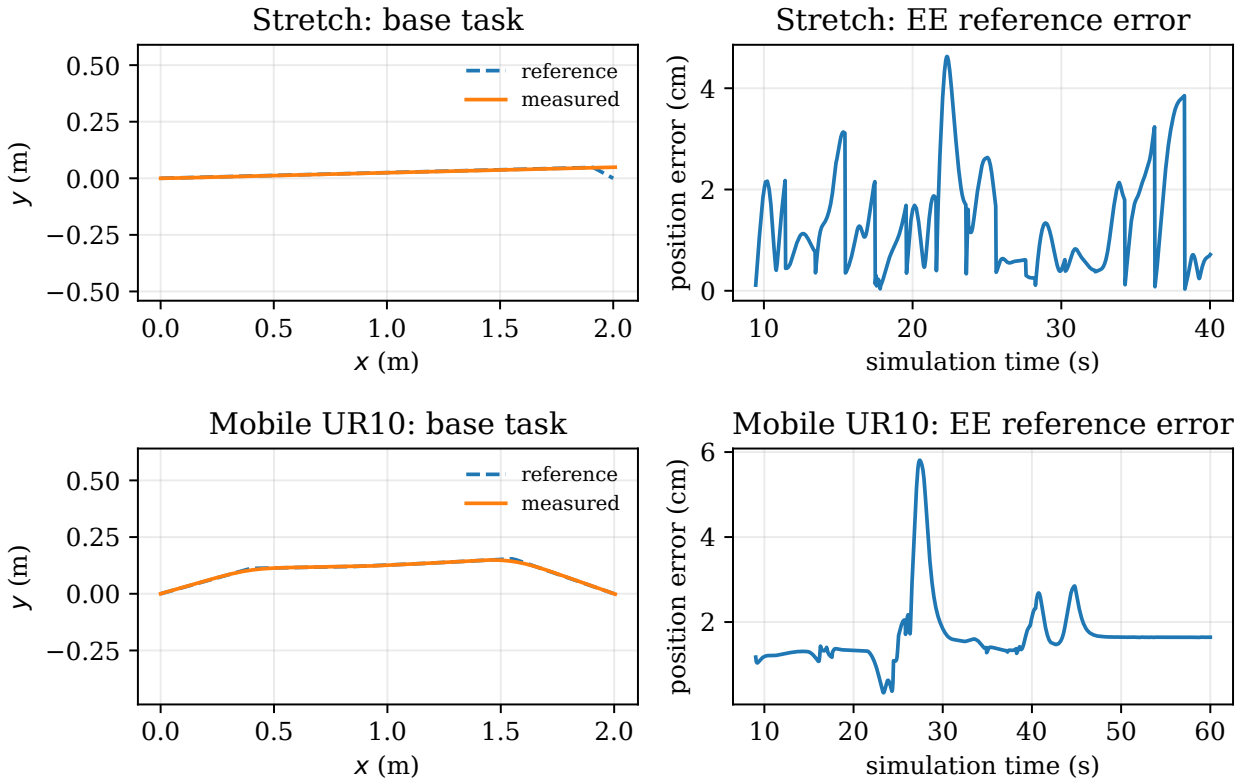

Fig. 4. Detailed tracking for both configurations. The left column shows reference and measured base paths during the base task; the right column shows Euclidean EE reference error during all subsequent task samples.

TABLE IV
RECORDED REAL-INTERFACE CHECKS

Check	Recorded result
Complete fused state	594 valid, 0 invalid
Base/joint timestamp skew	0 ms for all 594
Complete-state age	5.01 ms P99; 19.64 ms max
Concurrent command feedback	335 joint samples
Maximum feedback interval	41.218 ms
Base / wrist return error	0.670 mm / 7.03 mrad
Full WB-MPC hardware run	not performed

Both configurations tracked moving references to centimeter accuracy and used base motion during manipulation. This supports adaptability to the two tested mobile manipulators; it is not proof that an arbitrary robot works without a valid URDF, limits, sphere model, tuning, and state/command mapping.

Solver feasibility is platform-dependent and not equivalent to deployability. Stretch's P95 exceeded its 120-ms validity window, while the mobile UR10 had the lower P95 but still produced a single fallback and an approximately 0.8-s maximum. A synchronous simulator can continue after an overrun, whereas a real adapter must expire the old command and hold. The current guarded boundary implements that contract only for Stretch; a real UR10 deployment needs its own driver semantics, watchdog validation, and command-ownership checks. Improving real-time behavior may require fewer collision spheres, faster batched ESDF parameterization, tuned iteration limits, SQP-RTI, or a hard real-time fallback controller.

The collision result also needs careful interpretation. Positive 3.22- and 1.47-mm minimum margins mean the executed states stayed outside their configured boundaries in these static maps; neither is a robust physical clearance certificate. The offline simulated ESDF cannot observe a person or moved object in the real room. Map calibration and localization error can easily exceed a few millimeters, and the retained nominal controller URDF has documented reachable tool-frame differences up to 34.0 mm and 4.60 degrees relative to the deployed model. The real converter currently has no robot self-mask, and real depth is available as a comparatively low-rate keyframe stream. These facts prohibit treating the virtual ESDF as the only physical safety sensor.

The evaluation has additional limitations. It contains one deterministic trial per robot, no planner ablation, no sensor noise, and no dynamic obstacles; the durations and controller weights are not identical. Stretch's fourth task did not complete within its configured duration. Tracking error against a moving reference does not replace success rate across randomized scenes. The PyBullet input mapper currently exercises non-holonomic and omnidirectional bases, so documented fixed and floating modes are outside this evaluation. Finally, only Stretch

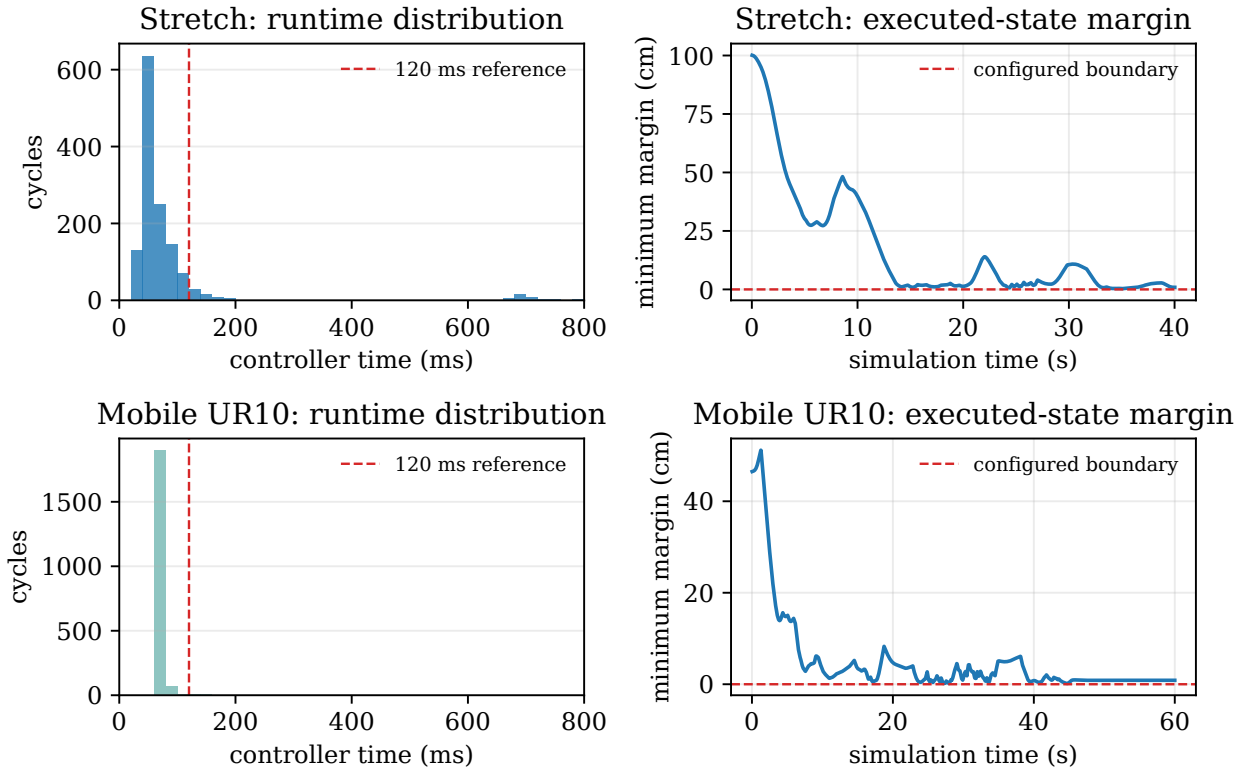

Fig. 5. Runtime distributions and minimum executed-state ESDF margins for Stretch (top) and mobile UR10 (bottom). The 120-ms line is a shared timing reference; the zero-margin line denotes each profile's own configured collision boundary.

has recorded hardware-interface tests, and its real profiles disable forward prediction because the predictor does not yet reproduce all adapter clipping and hold behavior.

VII. Conclusion

This work integrates ESDF-aware OMPL reference planning and configurable whole-body MPC for heterogeneous mobile manipulators. The shared pipeline ran with an 8-DoF nonholonomic Stretch model and a 9-DoF holonomic mobile-UR10 model, achieving 16.6- and 18.5-mm EE reference RMSE with valid ESDF queries and positive executed-state margins. Stretch-specific mimic mapping and a guarded ROS 2 adapter remain at the platform boundary rather than defining the framework. The timing tails and one mobile-UR10 fallback show that each new robot still requires timing and safety validation. Existing low-amplitude real tests validate only the Stretch state and driver contracts; full WB-MPC hardware execution and a real UR10 adapter remain future work.

The most important next steps are (i) online real-robot ESDF updates with robot self-filtering, (ii) solver-tail reduction and delay-consistent state prediction, and (iii) repeated shadow and guarded hardware trials across representative scenes, including an ESDF cost-versus-hard-constraint ablation.

Appendix A
Reproduction and Data Archive

The raw trials are stored without post-processing at:

```
results/report_experiments/
stretch_esdf_offline_ompl_wbmpc/
2026-08-14_12-25-30/raw/data.npz
results/report_experiments/
ur10_esdf_offline_ompl_wbmpc/
2026-08-14_12-51-35/raw/data.npz
```

Their SHA-256 values are

```
48556b2f89123df82d7f2519bc08eddf
60d5e9812194324e6daa9fc0803dc169
b67af19edba4d9cf3cd48228e1035f2
152dd7ed809613f7e490cc00abe51c23c.
```

Each adjacent `config.yaml` is the fully expanded runtime profile. From the repository root, metrics and figures are regenerated with:

```
pixi run python report/analyze_experiment.py
                <stretch>/raw/data.npz
                --ur10-data
<ur10>/raw/data.npz
```

The script writes `report/experiment_metrics.json` and all PDF figures. The real-interface evidence is recorded separately under `mm_run/results/real_state` and `mm_run/results/real_command`.

Appendix B
Procedure for Adding Another Robot

The two evaluated profiles suggest the following integration sequence for a third platform.

1) *Define the coordinate contract.* Provide simulation and controller joint names, $q/v/x/u$ dimensions, home state,

position, velocity, and acceleration bounds, tool and base frames, and a compilable URDF. Decide whether any URDF coordinates are mimic variables that should be reduced before optimization.

2) *Select and validate base semantics.* Choose the configured base type and verify forward/inverse state and command mappings separately. The current PyBullet path directly covers omnidirectional and nonholonomic bases; another actuation model requires a new simulator mapping even though the planning and MPC interfaces remain unchanged.
3) *Fit collision geometry and planning bounds.* Attach spheres to named robot links, select the active set, and choose safety distance, planner footprint, sampled heights, Cartesian bounds, and tool radius. Visual inspection should be followed by automated checks that every sphere frame exists and all nominal-state ESDF queries are valid.
4) *Generate and test the specialized solver.* A changed state or parameter dimension requires acados regeneration. First test stationary model consistency, then base-only and EE-only tracking, followed by the combined task sequence. Archive the expanded configuration and raw NPZ so failures can be reproduced without reconstructing include resolution.
5) *Treat hardware as a separate commissioning task.* Implement name-based feedback mapping, command conversion, freshness and timestamp checks, actuator-specific limits, watchdog behavior, and command ownership. Progress from read-only state validation to shadow control and bounded low-amplitude commands before enabling autonomous whole-body execution.

Successful simulation therefore establishes algorithmic compatibility, not hardware readiness. A new platform should be considered integrated only when its dimension and frame audits, collision coverage, task completion, solver tail, fallback behavior, and platform-specific stop path have all been measured under the intended operating conditions.

REFERENCES

[1] C. C. Kemp, A. Edsinger, H. M. Clever, and B. Matulevich, "The design of stretch: A compact, lightweight mobile manipulator for indoor human environments," in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3150–3157.

[2] H. Seraji, "A unified approach to motion control of mobile manipulators," *The International Journal of Robotics Research*, vol. 17, no. 2, pp. 107–118, 1998.

[3] A. De Luca, G. Oriolo, and P. Robuffo Giordano, "Kinematic control of nonholonomic mobile manipulators in the presence of steering wheels," in *2010 IEEE International Conference on Robotics and Automation*, 2010, pp. 1792–1798.

[4] M. Bhardwaj, B. Sundaralingam, A. Mousavian, N. D. Ratliff, D. Fox, F. Ramos, and B. Boots, "STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 750–759. [Online]. Available: https://proceedings.mlr.press/v164/bhardwaj22a.html

[5] C. Pezzato, C. Salmi, E. Trevisan, M. Spahn, J. Alonso-Mora, and C. H. Corbato, "Sampling-based model predictive control leveraging parallelizable physics simulations," *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 2750–2757, 2025.

[6] V. Vasilopoulos, S. Garg, P. Piacenza, J. Huh, and V. Isler, "RAMP: Hierarchical reactive motion planning for manipulation tasks using implicit signed distance functions," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 10 551–10 558.

[7] M. V. Minniti, F. Farshidian, R. Grandia, and M. Hutter, "Whole-body MPC for a dynamically stable mobile manipulator," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3687–3694, 2019.

[8] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *2012 IEEE International Conference on Robotics and Automation*, 2012, pp. 3859–3866.

[9] H. Oleynikova, Z. Taylor, M. Fehr, J. Nieto, and R. Siegwart, "Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1366–1373.

[10] A. Millane, H. Oleynikova, E. Wirbel, R. Steiner, V. Ramasamy, D. Tingdahl, and R. Siegwart, "nvblox: GPU-accelerated incremental signed distance field mapping," *arXiv preprint arXiv:2311.00626*, 2023.

[11] M. Gaertner, M. Bjelonic, F. Farshidian, and M. Hutter, "Collision-free MPC for legged robots in static and dynamic scenes," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 8266–8272.

[12] J.-R. Chiu, J.-P. Sleiman, M. Mittal, F. Farshidian, and M. Hutter, "A collision-free MPC for whole-body dynamic locomotion and manipulation," in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 4686–4693.

[13] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.

[14] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.